Hannah D'Cruz
Dr. Ming Li
CS 3083
6 Mar 2026

Game Tree Search and Constraint Satisfaction Problems

Task 1:

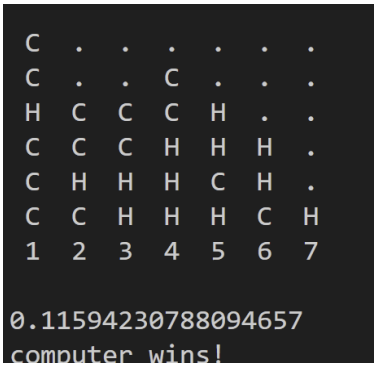
For Alpha-Beta search, the alpha-beta function is called once per turn. This calls the max function, which calls the min function, back and forth to build up a tree. If a max node finds a winner, it returns a very small value to encourage the min node to pick it. If a min node finds a winner, it returns a very small value to encourage the max node to pick it. If we have reached the cutoff depth, we have to evaluate the board to make our best guess who the winner would be or how close we are to finding a winner. This estimated value is between -2 and 2, because it is not as good or bad as an actual win or lose. Whatever value gets returned, we compare it to the value of the current node. Max nodes are looking for values bigger than their current value, and min nodes are looking for values smaller than their current value. If they find it, they update their own value and their own recommended move to correspond with the child node. If the value is not what we are looking for, but the max node's value is bigger than the current min node's value (stored in beta) or if the min node's value is smaller than the current max node's value (stored in alpha), it returns right away. This is because there is no point looking for a better value when the parent node will discard it anyway.

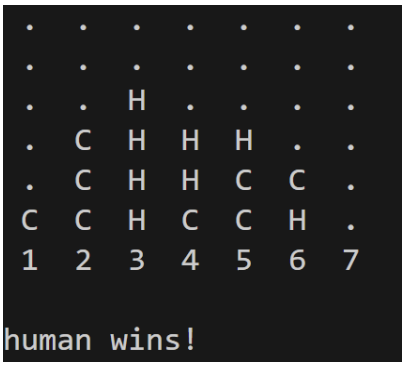
For my evaluation function, I iterate along every row, column, positive, and negative diagonal and count the number of consecutive pieces for each player, ignoring pieces that are not adjacent to another piece from the same player. I find the difference between the number of consecutive pieces for each player, then divide by 42 to get a number between -1 and 1. The more consecutive pieces this board option has, the higher the evaluation function, because it will be easier to win if you already have more pieces connected throughout the board. I chose to take the sum of all of them because you are most successful at connect 4 when you have multiple options for places to win so it is hard for your opponent to block all of them. If I had more time, I would improve my evaluation function by ignoring pieces that are already blocked off and not helpful anymore, because it is really easy to have three pieces in a column, but they are blocked by your opponent at the end and do not help your chances of winning.

As I increased my cutoff depth, the agent started playing better and better. This makes sense because the agent is able to look farther ahead and avoid backing itself into a corner where it will lose. After a cutoff depth of 5, the computer won, but the average time for my agent's turn increased significantly. Cutoff depth of 5 and 6 were about the same, but that is because the same was shorter for cutoff depth 5 than 6, so there were more long turns toward the beginning and fewer shorter turns once most of the board was filled and there were fewer places to search. Thus, I set my cutoff depth to 5.

Cutoff depth	Winner	Turns	Average agent turn length
2	Human	8	0.013032615184783936
3	Human	16	0.07615649700164795
4	Human	18	0.20266697141859266

5	Computer	17	1.2356734540727403
6	Computer	25	1.1633592752309947
7	Computer	19	6.319551968574524





(2) MCTS

For Monte Carlo Tree Search, the wrapper function creates a root node of a tree, with the current actual gameboard and no parents or children. Then it repeatedly calls MCTS a large number of times starting from the root each time. MTCS first checks if there are any columns (represented by children) of the current node that have never been simulated. If there are, it will simulate that position, add one to the playout for all its parents up to the root node, add one to all the nodes on the team that won (every other level in the tree), and then return. If the current node has expanded all its children, MCTS will instead use the UCB formula to pick which of its children it is going to travel down to, and continue travelling down using UCB until it finds a node with unexpanded children to simulate. After a preset number of simulations or hops down the tree, MCTS returns to the calling function. It does not need to return any data because the original head of the tree has been updated with all of the playout data. The calling function compares all the columns of the head node to pick the one with the highest number of playouts. It returns that column as the move the agent should make.

My random playout policy uses the numpy library to pick a random integer between 0 and 7 and tries to place its piece there on a temporary board, as long as the column is not full. If this creates a terminal state, the playout policy returns a 0, 0.5 or 1 for loss, tie, or win for the current node's player. Otherwise, it keeps looping to pick another random column and places the other player's piece on the temporary board. My win checking playout policy first tries placing its piece in every column to see if any will automatically make a winner. If so, it returns a win immediately. Otherwise, it checks to see if it needs to block an opponent from winning on the theoretical board, places that piece, and continues the playout. If neither case is true, it places a random piece and checks for a terminal state like the random policy.
It turns out my random playout policy performs slightly better than my checkWin policy, but that is because my checkWin policy is more likely to choose a starting position along one side, which leaves the start open for the human player to get three in a row with both sides open along the bottom of the board, ensuring a quick loss for the agent. After the initial couple plys, both policies work very well, and although the checkWin policy takes ten times as long as the random policy because it has to check the entire board for a winner so often, the time difference feels negligible to the user.

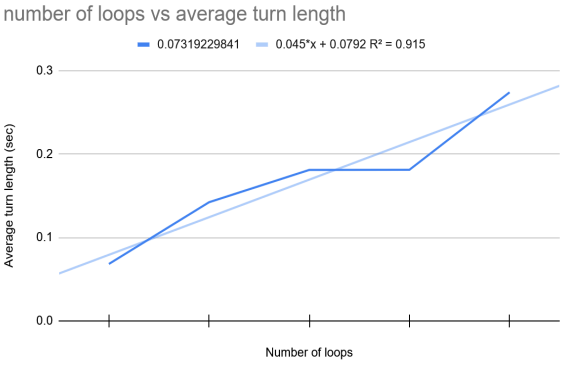
simulateCheckWin	Human win	8 turns	0.5436016321182251
simulateCheckWin	Computer win	17 turns	0.3376540607876248
simulateCheckWin	Human win	12 turns	0.4939194122950236
simulateRandom	Computer win	9 turns	0.0671119213104248
simulateRandom	Computer win	23 turns	0.04381189743677775
simulateRandom	Human win	8 turns	0.08240360021591187

I also tuned how many loops of MCTS were needed each turn to be most effective without being too slow. I measured how many turns it took for either the agent or the human to win, as well as the average length of the agent's turn. The graph shows that the average turn time increased roughly linearly compared to the number of loops, which makes sense because each loop should take about the same amount of time. However, a major limitation of this data is that I had to manually play against the agent each time, which took time and was highly dependent on the skill of the player (I got my friends to play some because I was bored of playing connect4). The data seems to show that a smaller amount of loops makes a more successful agent, but it was probably just that I was getting bored of playing. It could also be that my C value in the UCB was incorrect.

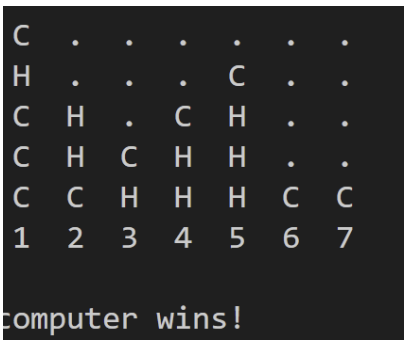
I later tuned the C value in 0.5 increments from 0.5 to 3.5. This also faced the problem of human variability, but the data seems to show that a value of approximately 1.5 is best.

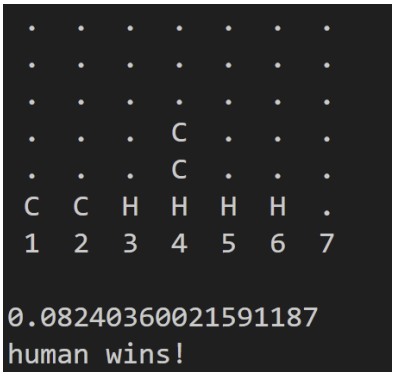
C	winner
0.5	human
0.5	human
1	human
1	computer
1.5	human
1.5	computer
2	computer
2	human
2.5	human
2.5	computer
3	human
3	computer
3.5	human
3.5	human

number of loops	Win %	Average time (sec)
100	100	0.06822926384
200	100	0.142171558
300	66	0.1809406164
400	66	0.1810701259
500	33	0.2738601318



Default c = 1.5
Default numLoops = 100 (bc fast)
Default simulateRandom



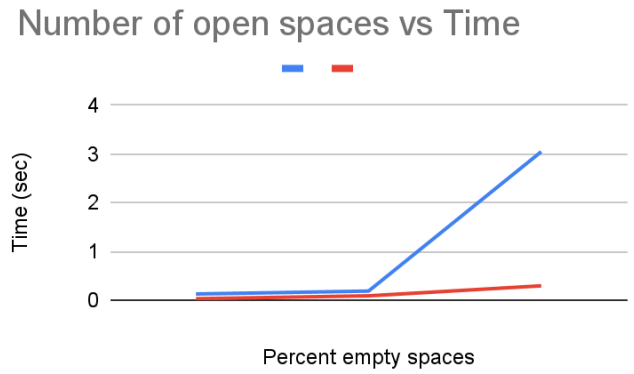


Task 2:
The basic sudoku solver program takes a board and calculates the domain for each open position. Then, it calls the backtracking method starting in the first row and column. The Backtrack method takes the board and a starting position as an input, and It checks as an input and checks if it's complete. If it's not, we find the next empty position to solve for. The original algorithm just iterates through the board in column major order to find the next blank position, but I also implemented the minimum remaining values heuristic. This chooses the position with the smallest domain as the next position to solve for, to hopefully avoid backtracking because we are less likely to pick an incorrect value. Once we have a position, we place the first value in its domain onto a temporary board. The original algorithm picks the smallest value, but to try even more to avoid backtracking later because of an empty domain, we also have the option of ordering the domain by least constraining value. This checks to see how many of the positions' neighbors' domains will be affected by placing each value in the position's domain, and putting the least impactful values first. Once we have assigned the first value in the domain to our position, we make a copy of the original board's domains and update it based on the newly filled position in the temporary board. We remove the value we just placed from all the position's neighbors' temporary domains. Forward-checking-only passes through all the neighbors once and then is done, but maintaining arc consistency keeps passing through all of the neighbors and re-updating their values based on the changes that were just made to neighbors' domains. If we find an empty domain while either forward checking or maintaining arc consistency, we immediately skip to the next value in this position's domain, discarding the temporary board and domain. Otherwise, we can continue on to the next position by recursively calling the backtrack function, passing in the temporary board and domain as the official one for the new function. Despite our best intentions, our program may still have to backtrack sometimes. If the backtrack function ever tries all the values in the domain without finding one that is allowed, it returns false. This causes the upper level to revert to the previous board and domain, and retry from the last unassigned position.
I stored my board as a 2D array, and stored the domains separately as a 3D array. In retrospect, the board array is likely redundant, since the domain of an assigned variable is reduced to just the value.
I collected about forty data points on a three by three boards with 20, 40, or 60 squares. My minimumRemainingValue heuristic improved the performance by about 50%. This makes sense, because even if we only avoided a few backtracks, every backtrack adds a significant amount of time to the program. My maintaining arc consistency was about 4 times worse than my forward checking only, which makes sense because maintaining arc consistency requires so many iterations through the sets of neighbors until none of them change. Least constraining value heuristic provided a small improvement, which makes sense because once you have already picked the function with the smallest domain, there are not many orders the values could be in, and fewer values means they are less likely to affect their neighbor's domains as well.

No MRV	84.03 sec	No MAC	24.57 sec	No LCV	64.31 sec
MRV	51.93 sec	MAC	109.50 sec	LCV	67.16 sec

My program works for all puzzle sizes. More open spaces correlates to more time. I did not get the 25x25 sudoku generator to run on my computer, so I was unable to test it with a puzzle.

3x3	25	0.03027629852	4x4	0.127430439
	50	0.08945790927		0.1870336533
	75	0.2950924635		3.04599762



Summary:
This project gave me a strong understanding of how to gradually improve algorithms with small improvements, well after the original algorithm is working. Even though the sudoku game function code took me a long time to write, implementing each part of the solver algorithm felt much more doable, and it was fun to compare different optimizations to see how they impacted the performance of the algorithm. Similarly, the monte carlo tree search took a bit to write out the basic functioning, but it was much more interesting to see how different parameters and playout policies influenced its performance, especially since I got to play against it and watch its steps in real time. I wish I had more time to spend tuning and optimizing parameters instead of spending so long figuring out how to set up and check a board. I preferred the data collection on sudoku because it was less variable.